Практикум студента

Сверка результатов и повторная корректировка витрины

Цель работы — исправить результат оконного расчёта после запоздавшей покупки и проверить повторное применение корректировки. Вы сопоставите предварительную таблицу с полным расчётом, объясните ошибку повторного прибавления и выполните замену строки по ключу и редакции.

Продолжение предыдущего задания

Используются те же 12 событий, пятиминутные окна и режим отставания watermark на две минуты. Предыдущие файлы результатов переносить не нужно. Новый проект заново воспроизводит исходный снимок 4800 и отдельно рассчитывает пакетный эталон 5500.

Этап самостоятельной работы	Время
Исходный снимок, ручные суммы и расхождение	20 минут
Неправильный повтор, корректировка и повторная проверка	20 минут
Защитные отказы, объяснения и упаковка	20 минут

До практики преподаватель объясняет маршрут 25 минут, после неё предусмотрено 5 минут обсуждения. Работайте индивидуально. Программа готова; собственные расчёты, прогнозы и объяснения обязательны. Оценивание проводит преподаватель.

Материалы и границы опыта

Нужны Windows 10/11 и Python 3.10 или новее, без дополнительных пакетов. После подготовки среды работа локальная. Это последовательная модель конечного CSV: распределённый кластер, реальный поток и сервер базы данных не запускаются.

В методичке команды и результаты показаны учебными иллюстрациями по проверенному выводу. Это не снимки рабочего стола Windows. Строку команды можно скопировать из текста под рисунком. Переходите к следующему этапу после проверки текущего. Результат сохраните локально для последующей сдачи в ЭОС.

1 Подготовка и место ввода команд

Скачайте стартовый ZIP и выберите в Проводнике «Извлечь всё». Откройте новую папку bigdata7, где рядом находятся lab.py, REPORT.txt и CMD-файлы. Не запускайте проект внутри ZIP и не распаковывайте его поверх предыдущей работы.

Дважды щёлкните 00_CHECK_ENV.cmd. Ожидается сообщение ниже. После чтения нажмите клавишу, чтобы закрыть окно. Если Python уже работал на предыдущем занятии, переустановка обычно не нужна.

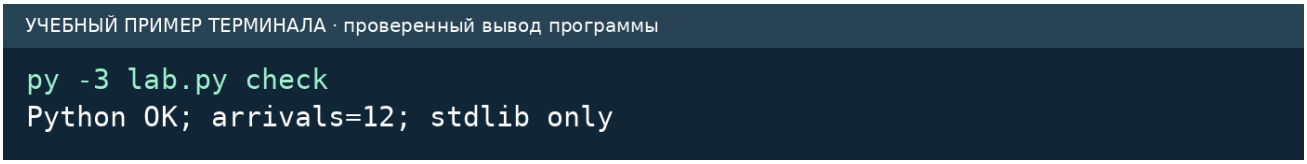


Рисунок 1. Среда найдена, источник содержит 12 событий.

```
py -3 lab.py check
```

Два способа одного запуска

Основной способ — двойной щелчок по очередному CMD-файлу. Резерв: в адресной строке Проводника этой папки введите cmd и нажмите Enter; вводите приведённые команды в командную строку Windows. Не вводите их в Блокнот, Git Bash или окно Python с приглашением >>>. Из >>> выйдите командой exit().

Если py не найден, используйте python lab.py check и заменяйте начало команд на python. Основной и резервный способы выполняют один этап; запускать оба подряд не требуется. При cannot open file lab.py откройте cmd заново именно из папки проекта.

Если Python отсутствует

Откройте DOWNLOADS.html с официальной ссылкой python.org/downloads/windows/. Нужен Python 3.10+. При запрете установки сообщите преподавателю и пока выполните ручные расчёты; техническую часть завершите после подготовки среды. Не отмечайте её как выполненную заранее.

2 Исходные события

Откройте data/arrivals.csv через «Открыть с помощью» → Блокнот. Можно использовать команду ниже в cmd. Все времена относятся к одному условному дню и единым часам. Не сортируйте и не редактируйте источник: порядок строк моделирует поступление.

УЧЕБНЫЙ ПРИМЕР ТЕРМИНАЛА · проверенный вывод программы

```
type data\arrivals.csv
event_id,event_time,arrival_time,event_type,status,amount
E001,10:01:00,10:01:00,purchase,success,100.00
E002,10:04:00,10:04:00,purchase,success,200.00
E003,10:05:00,10:05:00,purchase,success,300.00
E004,10:06:00,10:06:00,view,success,0.00
E005,10:03:00,10:07:00,purchase,success,400.00
E006,10:08:00,10:08:00,purchase,failed,50.00
E007,10:09:00,10:09:00,purchase,success,500.00
E008,10:12:00,10:12:00,purchase,success,600.00
E009,10:07:00,10:13:00,purchase,success,700.00
E010,10:10:00,10:14:00,purchase,success,800.00
E011,10:14:00,10:15:00,purchase,success,900.00
E012,10:16:00,10:16:00,purchase,success,1000.00
```

Рисунок 2. Те же исходные события, что в предыдущем задании.

`type data\arrivals.csv`

В суммы входят только purchase/success. E004 — просмотр, E006 — неуспешная покупка; они исключаются из сумм, но учитываются при продвижении максимального времени в прежней модели. Суммы источника записаны в рублях с двумя десятичными знаками.

Окна: [10:00,10:05), [10:05,10:10), [10:10,10:15), [10:15,10:20). Левая граница входит, правая не входит. Покупка ровно в 10:05 относится к окну с началом 10:05. E009 произошла в 10:07 и поступила в 10:13.

Перед запуском

Найдите три покупки второго окна: E003, E007 и E009. Выпишите их суммы и сложите вручную. Затем отметьте, какая из этих покупок была поздней при отставании границы на две минуты.

3 Воспроизведение предварительного снимка

Запустите 01_SNAPSHOT.cmd или выполните команду ниже. Откройте results/01_snapshot.txt. Предварительная витрина — сохранённая таблица итогов по окнам, которую позже требуется исправить.

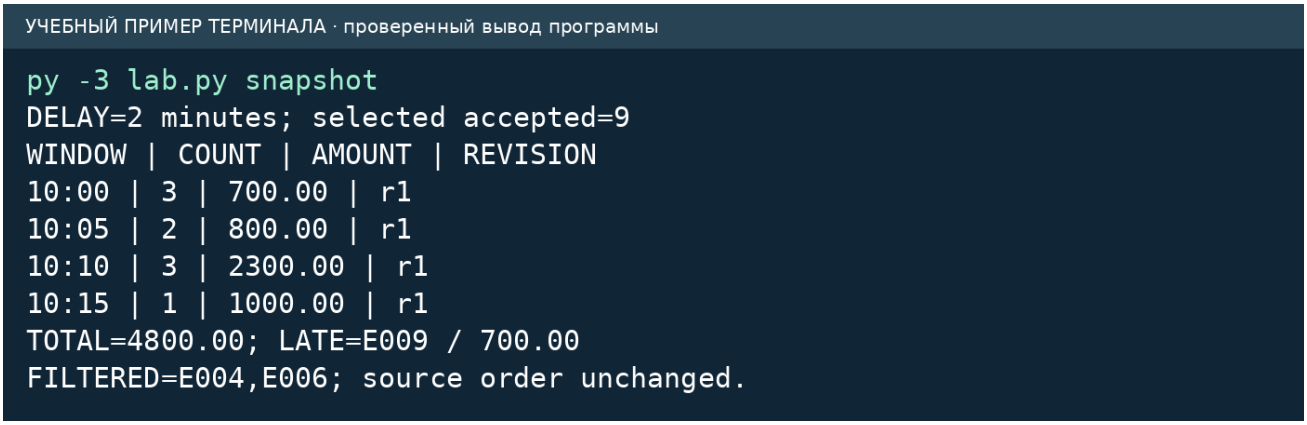


Рисунок 3. Режим двух минут воспроизводит прежний результат 4800.

```
py -3 lab.py snapshot
```

Проверьте: 9 принятых покупок, сумма 4800, поздняя E009 на 700. В results/preliminary.csv находятся четыре строки окон. Поле revision=1 обозначает исходную редакцию каждой строки результата.

Почему E009 отсутствует в снимке

До E009 максимальное увиденное event_time равно 10:12. При отставании две минуты watermark_before=10:10. Конец окна E009 равен 10:10. Прежнее правило проверяет конец окна ≤ watermark_before, поэтому покупка направляется в журнал поздних.

Исходное правило не меняется в этой работе. Новая задача — отдельно пересчитать и исправить таблицу результата. Не подменяйте условие в lab.py, чтобы E009 стала принятой исходным режимом.

Самопроверка

В results/trace.csv найдите E009 и решение LATE. E005 должна иметь ACCEPT. Сложите предварительные суммы 700 + 800 + 2300 + 1000. Запишите результат и объяснение E009 в поле 1 отчёта.

4 Пакетный эталон и ручные суммы

Запустите 02_REFERENCE.cmd. Программа читает весь известный исходник, фильтрует успешные покупки и группирует их по времени события без отбрасывания поздних записей. Откройте results/02_reference.txt и reference.csv.

УЧЕБНЫЙ ПРИМЕР ТЕРМИНАЛА · проверенный вывод программы

```
py -3 lab.py reference
BATCH REFERENCE: all 12 source rows; selected purchases=10
10:00 | 3 | 700.00 | r1
10:05 | 3 | 1500.00 | r1
10:10 | 3 | 2300.00 | r1
10:15 | 1 | 1000.00 | r1
TOTAL=5500.00
Finite-source reference; this does not close streaming windows.
```

Рисунок 4. Полный расчёт по конечному исходнику даёт 10 покупок и 5500.

`py -3 lab.py reference`

Рассчитайте самостоятельно

Для каждого из четырёх окон выпишите ID покупок, их количество и сумму. Во втором окне проверьте 300 + 500 + 700. В третьем учтите E010 ровно на левой границе 10:10. Не включайте просмотр и неуспешную покупку.

Общий итог равен 700 + 1500 + 2300 + 1000 = 5500. Эталонная таблица нужна для сравнения count и amount. Её revision=1 — служебная начальная отметка отдельной таблицы, а не номер уже выполненной корректировки витрины.

Что означает полнота

Эталон полон относительно предоставленных 12 строк. Это не утверждение о полноте будущего реального потока. В предыдущей модели конец файла не продвигал watermark и не закрывал OPEN-окна; здесь это правило также не изменяется.

Если источник случайно изменён, используйте новую распакованную копию и сохраните прежнюю работу отдельно. Не редактируйте контрольную сумму в source_manifest.json ради обхода проверки.

5 Поиск конкретного расхождения

Запустите 03_AUDIT.cmd. Прочитайте results/03_audit.txt. Сравнение строится по одинаковым ключам окон, фильтру и единицам измерения.

УЧЕБНЫЙ ПРИМЕР ТЕРМИНАЛА · проверенный вывод программы

```
py -3 lab.py audit
WINDOW | BEFORE | REFERENCE | DIFFERENCE
10:00 | 700.00 | 700.00 | 0.00
10:05 | 800.00 | 1500.00 | 700.00
10:10 | 2300.00 | 2300.00 | 0.00
10:15 | 1000.00 | 1000.00 | 0.00
MISMATCH: 10:05; missing count=1; missing amount=700.00
CAUSE: E009 was LATE under the previous rule.
```

Рисунок 5. Расхождение найдено в одном окне.

`py -3 lab.py audit`

Откройте results/late.csv и свяжите разницу с конкретной покупкой. Этот файл содержит заголовок и одну строку E009.

УЧЕБНЫЙ ПРИМЕР ТЕРМИНАЛА · проверенный вывод программы

```
type results\late.csv
event_id,event_time,arrival_time,event_type,status,amount
E009,10:07:00,10:13:00,purchase,success,700.00
```

Рисунок 6. Журнал поздних объясняет недостающие 700.

`type results\late.csv`

В отчёт запишите для окна 10:05: исходное количество и сумму, эталон и обе разницы. Не ограничивайтесь общей фразой «не хватало данных». Укажите ID и правило, по которому покупка не попала в предварительный результат.

Остальные окна уже совпали. Корректировка всего общего итога без указания окна не исправила бы распределение продаж по времени.

6 Чтение таблицы сверки

Откройте results/audit.csv в Блокноте или импортируйте в табличный редактор как UTF-8 с разделителем запятой. Не сохраняйте поверх исходного файла изменённые таблицы.

УЧЕБНЫЙ ПРИМЕР ТЕРМИНАЛА · проверенный вывод программы

```
type results\audit.csv
window_start,before_count,reference_count,delta_count,before_cents,refere
nce_cents,delta_cents
10:00,3,3,0,70000,70000,0
10:05,2,3,1,80000,150000,70000
10:10,3,3,0,230000,230000,0
10:15,1,1,0,100000,100000,0
```

Рисунок 7. Разницы количества и суммы вычислены для каждого окна.

`type results\audit.csv`

Поле	Как читать
before_count / reference_count	Количество покупок в снимке и полном расчёте.
delta_count	Эталонное количество минус предварительное.
before_cents / reference_cents	Суммы в копейках, а не в рублях.
delta_cents	Разница сумм: 70000 копеек означает 700 рублей.

Для строки 10:05 проверьте расчёт 150000 − 80000 = 70000 копеек. Для других строк обе разницы нулевые. Денежные значения программа хранит целым числом копеек, чтобы сравнение не зависело от округления двоичных дробей.

Почему сверяется каждое окно

Представьте ошибку: в первом окне лишние 100 рублей, во втором недостающие 100. Общая сумма не изменится, но строки неверны. Поэтому критерий корректности включает count и amount для каждого ключа. Запишите этот пример своими словами в отчёт.

7 Неправильный повтор прибавления

До запуска запишите прогноз: что получится после одного и после двух прибавлений E009 к предварительным 4800? Затем запустите 04_NAIVE.cmd и откройте results/04_naive.txt.

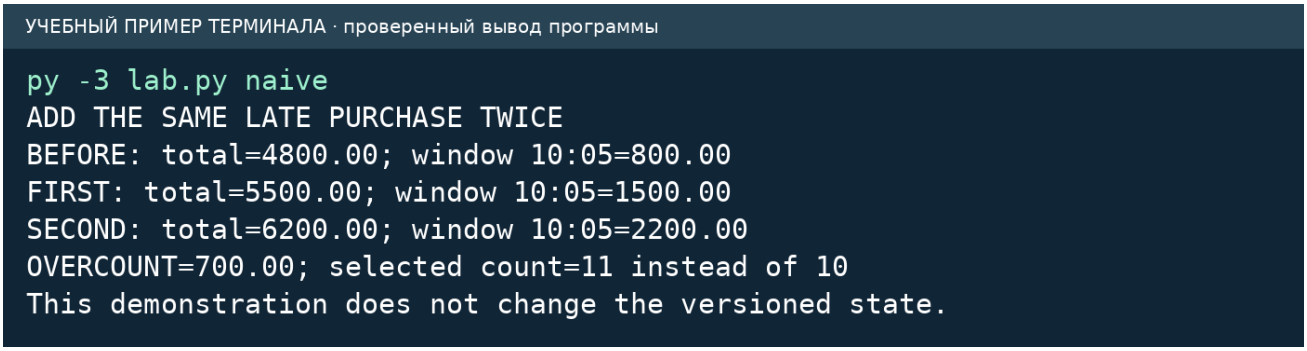


Рисунок 8. Повтор прибавления завышает сумму и количество.

```
py -3 lab.py naive
```

Первое прибавление даёт 5500, второе — 6200. Сумма окна 10:05 меняется 800 → 1500 → 2200. Количество успешных покупок после второго прибавления становится 11 вместо 10.

Объясните ошибку

Исходных событий по-прежнему 12. Новая покупка не появилась. Повторно применено одно и то же исправление E009 на 700. Ошибка находится в способе обновления результата, который каждый раз прибавляет значение без учёта предыдущего применения.

Программа сохраняет naive_once.csv и naive_twice.csv отдельно. Эти демонстрационные таблицы не изменяют состояние защищённой витрины, с которой будут работать apply и repeat. Неправильный итог нужно объяснить, а не исправлять вручную в CSV.

Задание в отчёт

Сопоставьте свой прогноз с выводом. Запишите лишнюю сумму, повторно учтённый ID и неверное количество. Объясните, почему одного успешного первого запуска недостаточно для проверки повторяемой операции.

8 Ключ и полное значение корректировки

Запустите 05_PATCH.cmd. Исправление пересчитывается из пакетного эталона только для строки с расхождением. Откройте results/05_patch.txt и patch.csv.

УЧЕБНЫЙ ПРИМЕР ТЕРМИНАЛА · проверенный вывод программы

```
py -3 lab.py patch
REPLACEMENT PATCH
KEY=10:05; window=[10:05,10:10)
COUNT=3; AMOUNT=1500.00; REVISION=2
Payload contains the FULL corrected window value, not +700.
Only this window is changed. Other rows retain revision 1.
```

Рисунок 9. Корректировка передаёт полное исправленное значение окна.

```
py -3 lab.py patch
```

УЧЕБНЫЙ ПРИМЕР ТЕРМИНАЛА · проверенный вывод программы

```
type results\patch.csv
window_start,count,sum_cents,revision
10:05,3,150000,2
```

Рисунок 10. В CSV сумма выражена целым числом копеек.

```
type results\patch.csv
```

Ключ window_start=10:05 находит строку. Новые значения: count=3, sum_cents=150000, revision=2. Полная сумма — 1500 рублей. Число 700 относится только к разнице с предыдущим результатом и не является содержимым замены.

Короткий ключ допустим здесь для одного дня, одного потока и фиксированных окон. Для нескольких дней или категорий продаж потребуются дополнительные части ключа. Номер редакции означает последовательность согласованных результатов этого окна, а не event_time или время запуска компьютера.

Объясните две ошибки: замена окна на 700 теряет ранее учтённые 800; прибавление полного значения 1500 к 800 даёт лишнюю сумму. В выбранном способе строка заменяется целиком.

9 Первое применение корректировки

Запустите 06_APPLY.cmd. Программа начинает с предварительной таблицы и применяет новую редакцию. Откройте results/06_apply.txt и after_first.csv.

УЧЕБНЫЙ ПРИМЕР ТЕРМИНАЛА · проверенный вывод программы

```
py -3 lab.py apply
KEY=10:05; ACTION=REPLACE
BEFORE: count=2; amount=800.00; revision=1
AFTER: count=3; amount=1500.00; revision=2
TOTAL=5500.00; selected count=10
Saved results/state.json for the next command.
```

Рисунок 11. Новая редакция заменяет одну строку и сохраняет состояние.

```
py -3 lab.py apply
```

Действие REPLACE означает замену: редакция 2 новее исходной редакции 1. После неё окно 10:05 содержит три покупки и 1500. Общая сумма равна 5500, количество — 10. Остальные окна не меняются.

Где сохранён результат

results/state.json — текущее состояние для следующего запуска. results/after_first.json и after_first.csv — снимки первого исправленного результата. Не редактируйте эти файлы: следующая команда должна проверить настоящий результат первого применения.

Если случайно запустили apply ещё раз после успешного выполнения, программа сообщит, что состояние уже исправлено, и предложит перейти к repeat. Она не прибавляет новую сумму и не перезаписывает первоначальное наблюдение.

Перед повтором

В отчёт запишите ключ, действие, исходную и новую редакции, сумму окна и общий итог. Затем сформулируйте прогноз для повторного применения той же корректировки к сохранённому исправленному состоянию.

10 Повтор на сохранённой витрине

Запустите 07_REPEAT.cmd. Этот этап читает state.json, созданный первым применением. Он не начинает заново с предварительного снимка. Откройте results/07_repeat.txt.

УЧЕБНЫЙ ПРИМЕР ТЕРМИНАЛА · проверенный вывод программы

```
py -3 lab.py repeat
READ: saved results/state.json
KEY=10:05; ACTION=NOOP_SAME
TOTAL_BEFORE=5500.00; TOTAL_AFTER=5500.00
UNCHANGED=True; count=10; window revision=2
Same correction repeated on the persisted state.
```

Рисунок 12. Точный повтор получает NOOP_SAME и оставляет состояние прежним.

`py -3 lab.py repeat`

УЧЕБНЫЙ ПРИМЕР ТЕРМИНАЛА · проверенный вывод программы

```
type results\after_repeat.csv
window_start,window_end,count,amount,revision
10:00,10:05,3,700.00,1
10:05,10:10,3,1500.00,2
10:10,10:15,3,2300.00,1
10:15,10:20,1,1000.00,1
```

Рисунок 13. Исправленная строка имеет редакцию 2, остальные сохраняют редакцию 1.

`type results\after_repeat.csv`

Сравните after_first.csv и after_repeat.csv: ключи, количества, суммы и редакции должны совпасть. NOOP_SAME означает совпадение номера и содержимого корректировки с уже сохранённой строкой.

Идемпотентность в этом опыте: повтор того же исправления не меняет уже полученное состояние. Это конкретное свойство операции обновления; оно не означает, что программа вообще никогда не выполняет работу повторно.

В отчёте укажите, что повтор проверен именно на сохранённом состоянии. Одно совпадение общего итога после двух независимых расчётов с нуля не было бы такой же проверкой.

11 Устаревшая и конфликтующая корректировки

Запустите 08_GUARDS.cmd. Он создаёт две контрольные попытки на копии исправленного состояния. Рабочий state.json не меняется. Откройте results/08_guards.txt.

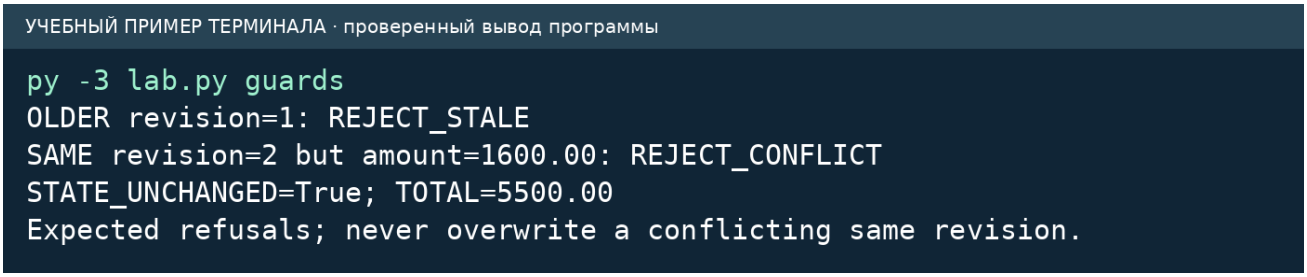


Рисунок 14. Обе нежелательные корректировки отклонены с разными причинами.

`py -3 lab.py guards`

Попытка	Решение и причина
Редакция 1 после принятой 2	REJECT_STALE: старый результат не возвращается поверх нового.
Редакция 2 с суммой 1600	REJECT_CONFLICT: тот же номер сопровождается иным содержимым.
Редакция 2 с прежними 1500	NOOP_SAME: точный повтор уже принятой строки.

Первые два случая сохраняют 5500. Отказ является ожидаемым результатом проверки, а не признаком неисправности проекта. Запишите оба решения и объясните, почему они отличаются.

Конфликтующая сумма не становится правильной только потому, что больше или пришла позже. Нужен разбор источника корректировки. В модели редакции выдаются в заранее согласованном порядке; программа сама не проверяет достоверность расчёта по одному номеру.

12 Итоговая сверка и границы вывода

Запустите 09_COMPARE.cmd. Проверяются исходный снимок, все эксперименты, содержимое CSV и JSON, а также соответствие исправленных значений пакетному эталону по каждому окну.

УЧЕБНЫЙ ПРИМЕР ТЕРМИНАЛА · проверенный вывод программы

```
py -3 lab.py compare
PASS: every corrected window matches batch count and amount.
PASS: replay on saved state leaves 5500.00 unchanged.
PASS: stale and conflicting corrections leave state unchanged.
NAIVE SECOND=6200.00; VERSIONED REPEAT=5500.00
This finite sequential test is not an exactly-once guarantee.
```

Рисунок 15. Проверка подтверждает конкретные свойства учебного опыта.

`py -3 lab.py compare`

Итог: первое исправление и его повтор дают 5500, неправильный второй ADD — 6200. Для каждого окна count и amount исправленной витрины совпадают с эталоном. Редакции не обязаны совпадать с редакциями отдельной эталонной таблицы.

Что этот опыт не доказывает

Нет распределённых узлов, конкурирующих записей, аварии посреди сохранения и транзакционной связи с источником сообщений. Поэтому результат не является гарантией exactly-once всей системы. Он подтверждает последовательную идемпотентную корректировку в данном локальном опыте.

Корректировка витрины не меняет прежний watermark. Окно, которое прежнее правило считало закрытым, может получить отдельную пакетную редакцию отчётной строки. Это другой процесс; не называйте его автоматическим повторным открытием окна в Spark или Flink.

Если проверка остановилась

Missing означает пропущенный этап; выполните его по порядку. Changed означает изменённый файл результата. Вернитесь к соответствующему этапу и восстановите таблицу расчётом. Не исправляйте суммы вручную ради PASS.

13 Отчёт по собственным наблюдениям

Откройте REPORT.txt в Блокноте. Замените каждую пометку [заполнить] своим ответом. Сохраните UTF-8 и исходное имя. Включите показ расширений файлов в Проводнике, чтобы не получить REPORT.txt.txt.

Поля отчёта	Основание для ответа
1–3: исходник, окна и расхождение	Ваши ручные суммы, preliminary, reference, audit и late.
4–7: неправильный повтор и замена	Прогноз, naive_once, naive_twice и patch.
8–10: повтор и защитные отказы	after_first, after_repeat, state и guards.
11–14: сверка и итог	compare, собственные объяснения и границы модели.

Пример формы объяснения

«В окне … было … покупок на … . Эталон показывает … . Расхождение связано с событием … . Первое применение выполнило … , повторное — … . Состояние … , что подтверждается файлами …». Подставьте собственные наблюдения; пример не заменяет остальные ответы.

Для идемпотентности укажите повтор той же корректировки на сохранённой таблице. Для двух отказов объясните номер редакции и содержимое. Для ограничений назовите хотя бы два условия распределённой обработки, которых в опыте не было.

Проверка перед упаковкой

Все обязательные этапы выполнены; таблицы не редактировались вручную; ответы содержат суммы и причины; технические затруднения описаны честно. Самопроверку выполняете вы, содержание отчёта оценивает преподаватель.

Если техническая часть ещё не завершена, сохраните заполненные ответы и рабочую папку. Не придумывайте результаты; завершите недостающие этапы до объявленного срока сдачи.

14 Архив и последующая загрузка

После заполнения отчёта запустите 10_PACK.cmd. Программа повторит сверку и создаст архив автоматически. Упаковывать весь проект вручную не нужно.

УЧЕБНЫЙ ПРИМЕР ТЕРМИНАЛА · проверенный вывод программы

```
py -3 lab.py pack
PASS: every corrected window matches batch count and amount.
PASS: replay on saved state leaves 5500.00 unchanged.
PASS: stale and conflicting corrections leave state unchanged.
NAIVE SECOND=6200.00; VERSIONED REPEAT=5500.00
This finite sequential test is not an exactly-once guarantee.
READY: submission/BigData7_result.zip
```

Рисунок 16. READY сообщает относительный путь к архиву результата.

`py -3 lab.py pack`

Откройте submission/BigData7_result.zip в Проводнике. Проверьте наличие REPORT.txt, lab.py, source_manifest.json, data/arrivals.csv и папки results с таблицами и наблюдениями. Упаковка проверяет структуру и вычисления, но не оценивает смысл ответов.

Сохраните ZIP и исходную рабочую папку до проверки преподавателем. Позже загрузите один BigData7_result.zip в задание ЭОС по объявленной ссылке. Если после загрузки требуется окончательная отправка, подтвердите её соответствующей кнопкой.

Частые сообщения

Fill REPORT.txt: остались подсказки или слишком короткий отчёт. Missing or changed: пропущен этап или изменён результат. Source changed: нарушен исходный CSV. При вопросе сообщите этап, точную команду и полный вывод.

Если окно закрывается слишком быстро, используйте ручной запуск через cmd из папки проекта. Если текст CSV открылся одним столбцом, импортируйте его с разделителем запятой и UTF-8. Не сохраняйте перекодированную таблицу поверх оригинала.

15 Критерии и контрольные вопросы

Критерий	Полное выполнение на 2 балла
Исходное состояние и эталон	Верно рассчитаны окна, объяснены 4800, 5500 и роль E009.
Ошибка повторного прибавления	Есть прогноз, получены 5500 и 6200, объяснён повторный учёт 700.
Ключ и полная замена	Различены ключ, полное значение 1500, разница 700 и редакция результата.
Повтор и защиты	Проверены REPLACE, NOOP_SAME, REJECT_STALE и REJECT_CONFLICT; объяснено неизменное состояние.
Сверка и вывод	Сопоставлены все окна, указаны ограничения опыта, заполнен отчёт и подготовлен ZIP.

За каждый критерий: 2 — верно и объяснено; 1 — частично или объяснение неполное; 0 — отсутствует или неверно. Максимум 10 баллов. Оценивание выполняет преподаватель.

Контрольные вопросы

1. Почему E009 не входит в предварительные 4800, но входит в пакетные 5500?
2. Чем отличаются разница 700 и полное значение окна 1500?
3. Почему для проверки повтора важно прочитать сохранённое состояние?
4. Чем REJECT_STALE отличается от REJECT_CONFLICT?
5. Почему корректные общая сумма и один запуск не доказывают exactly-once?

Дополнительно для завершивших раньше

В lab.py найдите функцию replace. Прочитайте порядок сравнений revision и значений. Объясните, что произошло бы при безусловной замене любой пришедшей строкой. Код основной попытки не меняйте. Предложите состав ключа для нескольких дней и магазинов.

Работа использует собственное явно заданное правило обновления. Понятие замены по ключу связано с upsert, но PostgreSQL или другая СУБД в проекте не запускается.